

Data Visualization — Complete Chart Reference

Matplotlib · Seaborn · Plotly | When to use · Code · Avoid when · Equivalents

Library	Best for	Key strength
Matplotlib	Custom / publication-ready static plots	Full control over every pixel of the chart
Seaborn	Statistical analysis & EDA with DataFrames	Beautiful defaults, built-in stats (CI, regression)
Plotly	Interactive dashboards & web/browser charts	Hover, zoom, pan, animations, maps out of the box

■ Rule of thumb: Use **Matplotlib** for papers/reports. Use **Seaborn** for pandas EDA. Use **Plotly** for dashboards, web apps, or anything interactive.

① Matplotlib — plt.*

The foundation of Python plotting. Maximum control. Used directly or under the hood by Seaborn.

plt.plot() — Line Chart

When to use

Show continuous change over time or ordered data. Default go-to for time series.

■ Avoid when

X-axis is categorical — use bar chart instead.

↔ Equivalent

sns.lineplot() · px.line()

```
import matplotlib.pyplot as plt
plt.plot(x, y, color='steelblue', linewidth=2, marker='o')
plt.title('Monthly Revenue') ; plt.xlabel('Month') ; plt.show()
```

plt.bar() / plt.barh() — Bar Chart

When to use

Compare quantities across discrete categories. Most-used chart for categorical data.

■ Avoid when

More than 15 categories — chart becomes unreadable.

↔ Equivalent

sns.barplot() · px.bar()

```
plt.bar(categories, values, color='steelblue', edgecolor='white')
# Horizontal: plt.barh(categories, values)
```

plt.scatter() — Scatter Plot

When to use

Show relationship/correlation between 2 numeric variables. Find patterns, clusters, outliers.

■ Avoid when

Categorical axes — use bar or box plot.

↔ Equivalent

sns.scatterplot() · px.scatter()

```
plt.scatter(x, y, c=color_var, s=size_var, alpha=0.6, cmap='viridis')
plt.colorbar() ; plt.xlabel('Height') ; plt.ylabel('Weight')
```

plt.hist() — Histogram

When to use

Show frequency distribution of a single numeric variable. See where values cluster.

■ Avoid when

Categorical data — use bar chart instead.

↔ Equivalent

sns.histplot() · px.histogram()

```
plt.hist(data, bins=30, color='steelblue', edgecolor='white', density=False)
# density=True → shows probability density instead of count
```

plt.boxplot() — Box Plot

When to use

Show median, IQR (25–75%), whiskers, and outliers. Great for comparing spread across groups.

■ Avoid when

Need to see exact distribution shape — use violin plot.

↔ Equivalent

sns.boxplot() · px.box()

```
plt.boxplot([group_a, group_b, group_c], labels=['A','B','C'], patch_artist=True)
# patch_artist=True fills boxes with color
```

plt.pie() — Pie / Donut Chart

When to use

Show proportions of a whole. Only effective with ≤5 clearly different slices.

■ Avoid when

More than 5 slices or when comparing multiple pies — use bar chart.

↔ Equivalent

px.pie() · (seaborn has no native pie)

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
# Donut: add wedgeprops=dict(width=0.5)
```

plt.fill_between() — Area Chart

When to use

Show volume/magnitude of a trend.
Emphasize cumulative totals over time.

■ Avoid when

Multiple overlapping series — becomes messy.

↔ Equivalent

sns.lineplot() + fill_between · px.area()

```
plt.plot(x, y) ; plt.fill_between(x, y, alpha=0.3, color='steelblue')
# Stacked: use plt.stackplot(x, y1, y2, y3, labels=[...])
```

plt.imshow() — Heatmap / Image

When to use

Display 2D matrix data as a color grid.
Great for correlation matrices, confusion matrices.

■ Avoid when

Few rows/cols where a grouped bar chart is clearer.

↔ Equivalent

sns.heatmap() (much easier) · px.imshow()

```
plt.imshow(matrix, cmap='coolwarm', aspect='auto')
plt.colorbar() ; plt.xticks(range(n), col_labels, rotation=45)
```

plt.contour() / plt.contourf() — Contour Plot

When to use

Show 3D surface data in 2D using contour lines. Common in ML decision boundaries.

■ Avoid when

Simple 2-variable data — scatter is clearer.

↔ Equivalent

px.contour() / px.density_contour()

```
plt.contourf(X, Y, Z, levels=20, cmap='RdYlBu')
plt.colorbar() # filled contour is more readable than lines only
```

plt.errorbar() — Error Bar Plot

When to use

Show mean ± standard deviation or confidence interval. Important in scientific plotting.

■ Avoid when

When exact individual values matter — use scatter or strip plot.

↔ Equivalent

sns.pointplot() (with CI) · px.scatter() with error_y

```
plt.errorbar(x, means, yerr=std_devs, fmt='o', capsize=5, color='steelblue')
# yerr can be symmetric (one value) or asymmetric ([lower, upper])
```

plt.stackplot() — Stacked Area Chart

When to use

Show how multiple series add up to a total over time. Good for proportional evolution.

■ Avoid when

Negative values — stacked area breaks visually.

↔ Equivalent

px.area(groupby) · (seaborn: manual with fill_between)

```
plt.stackplot(x, y1, y2, y3, labels=['A', 'B', 'C'], alpha=0.8)
plt.legend(loc='upper left')
```

plt.step() — Step Chart

When to use

Show data that changes at discrete moments, not continuously.

■ Avoid when

Continuously changing data — line chart is more accurate.

↔ Equivalent

`px.line(line_shape='hv') · sns.lineplot()`
with `drawstyle`

```
plt.step(x, y, where='post', color='steelblue')  
# where='post': step after each point | 'pre': before | 'mid': middle
```

② Seaborn — sns.*

Built on Matplotlib. DataFrame-native. Beautiful statistical charts with minimal code.

sns.lineplot() — Line Plot

When to use

Line chart with automatic confidence interval bands. Best when you have repeated measurements per x.

■ Avoid when

Single clean time series — `plt.plot()` is simpler and faster.

↔ Equivalent

`plt.plot() · px.line()`

```
import seaborn as sns
sns.lineplot(data=df, x='month', y='revenue', hue='region', ci=95)
# ci=95 draws 95% confidence interval shading automatically
```

sns.barplot() — Bar Plot

When to use

Bar chart with mean + confidence interval per category. Ideal for statistical comparisons.

■ Avoid when

Just showing raw totals — `plt.bar()` or `px.bar()` is more direct.

↔ Equivalent

`plt.bar() · px.bar()`

```
sns.barplot(data=df, x='city', y='sales', hue='year', palette='Blues_d')
# hue splits bars into sub-groups (grouped bar chart)
```

sns.countplot() — Count Plot

When to use

Count frequency of each category automatically. No need to pre-aggregate.

■ Avoid when

You already have aggregated counts — use `barplot()` instead.

↔ Equivalent

`df[col].value_counts().plot(kind='bar') · px.histogram()`

```
sns.countplot(data=df, x='category', order=df['category'].value_counts().index)
# order= sorts bars by frequency (most common first)
```

sns.histplot() — Histogram

When to use

Distribution of numeric data with optional KDE overlay. More powerful than `plt.hist()`.

■ Avoid when

Categorical data — use `countplot()`.

↔ Equivalent

`plt.hist() · px.histogram()`

```
sns.histplot(data=df, x='age', bins=30, kde=True, color='steelblue')
# kde=True overlays smooth density curve on the histogram
```

sns.kdeplot() — KDE Plot

When to use

Smooth continuous distribution curve. Shows the shape without the binning artifact of histograms.

■ Avoid when

Small datasets (<30) — the curve can be misleading.

↔ Equivalent

`sns.histplot(kde=True) · px.violin()` (partial)

```
sns.kdeplot(data=df, x='salary', hue='department', fill=True, alpha=0.4)
# fill=True shades under the curve for easier comparison
```

sns.boxplot() — Box Plot

When to use

Median, IQR, whiskers, and outlier dots per group. Great for comparing spread across categories.

■ Avoid when

Need exact distribution shape — use violin plot.

↔ Equivalent

`plt.boxplot()` · `px.box()`

```
sns.boxplot(data=df, x='grade', y='score', palette='Set2', width=0.5)
# width=0.5 makes boxes narrower and easier to compare
```

sns.violinplot() — Violin Plot

When to use

Box plot + KDE shape combined. See distribution density AND outliers at once.

■ Avoid when

Small datasets — violin shape is unreliable with <30 points.

↔ Equivalent

`px.violin()` · (matplotlib: manual)

```
sns.violinplot(data=df, x='class', y='age', hue='sex', split=True, palette='muted')
# split=True shows two hues on each side of the violin
```

sns.stripplot() / sns.swarmplot() — Strip & Swarm

When to use

Plot every individual data point. Swarm avoids overlap. Perfect for small/medium datasets.

■ Avoid when

Large datasets (>500 pts) — swarm becomes too crowded.

↔ Equivalent

`px.strip()` · (matplotlib: `plt.plot` with jitter)

```
sns.swarmplot(data=df, x='group', y='value', palette='Set2', size=4)
# Combine with boxplot: sns.boxplot() first, then sns.stripplot(alpha=0.4)
```

sns.scatterplot() — Scatter Plot

When to use

Relationship between 2 numeric vars with easy hue/size/style encoding for extra dimensions.

■ Avoid when

Categorical axes — use barplot.

↔ Equivalent

`plt.scatter()` · `px.scatter()`

```
sns.scatterplot(data=df, x='height', y='weight', hue='gender', size='age',
sizes=(20, 200), palette='Set1', alpha=0.7)
```

sns.regplot() / sns.lmplot() — Regression Plot

When to use

Scatter + linear regression line + confidence interval. Instantly shows trend direction.

■ Avoid when

Non-linear relationships — the straight line will be misleading.

↔ Equivalent

`px.scatter(trendline='ols')` · (matplotlib: manual `polyfit`)

```
sns.regplot(data=df, x='study_hours', y='grade', scatter_kws={'alpha':0.5})
# lmplot() = regplot + FacetGrid for multiple groups by hue/col/row
```

sns.heatmap() — Heatmap

When to use

Color-coded 2D matrix. Best for correlation matrices, confusion matrices, pivot tables.

■ Avoid when

Non-matrix data — use grouped bar chart.

↔ Equivalent

`plt.imshow()` · `px.imshow()`

```
corr = df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, square=True)
```

sns.pairplot() — Pair Plot

When to use

Scatter matrix for all numeric variable pairs + diagonal histograms/KDE. Essential for EDA.

■ Avoid when

More than 7 features — plots become too small to read.

↔ Equivalent

`px.scatter_matrix()` · (matplotlib: manual subplots)

```
sns.pairplot(df, hue='species', diag_kind='kde', plot_kws={'alpha':0.5})
# hue colors all plots by a category | diag_kind='kde' for density on diagonal
```

sns.FacetGrid — Facet Grid

When to use

Create a grid of subplots automatically split by a column/row categorical variable.

■ Avoid when

Simple single-group visualization — overkill without multiple categories.

↔ Equivalent

`px.facet_col()` / `px.facet_row()` · `plt.subplots()`

```
g = sns.FacetGrid(df, col='department', row='year', height=3)
g.map(sns.histplot, 'salary', bins=20) ; g.add_legend()
```

sns.jointplot() — Joint Plot

When to use

Scatter in center + marginal distributions on x and y axes. Great for bivariate analysis.

■ Avoid when

More than 2 numeric variables — use pairplot.

↔ Equivalent

`px.scatter()` with `marginal_x/marginal_y` · (matplotlib: manual GridSpec)

```
sns.jointplot(data=df, x='height', y='weight', kind='reg', marginal_kws={'bins':20})
# kind: 'scatter' | 'kde' | 'hist' | 'hex' | 'reg' | 'resid'
```

sns.clustermap() — Cluster Heatmap

When to use

Heatmap with hierarchical clustering on both rows and columns. Used in bioinformatics/ML.

■ Avoid when

Small datasets or when ordering is meaningful — regular heatmap is clearer.

↔ Equivalent

`px.imshow()` with clustering · (matplotlib: manual dendrograms)

```
sns.clustermap(df_pivot, cmap='vlag', figsize=(10,8),
z_score=1, method='ward') # z_score=1 normalizes rows
```

③ Plotly Express + Graph Objects — px.* / go.*

Interactive charts for the browser. Hover, zoom, maps, 3D — all built-in. Best for dashboards.

px.line() — Interactive Line Chart

When to use

Time series or ordered data with hover, zoom, pan built-in. Best for dashboards/web.

■ Avoid when

Static reports/papers — matplotlib is simpler and lighter.

↔ Equivalent

plt.plot() · sns.lineplot()

```
import plotly.express as px
fig = px.line(df, x='date', y='revenue', color='region', title='Monthly Revenue')
fig.show() # or fig.write_html('chart.html') to save
```

px.bar() — Interactive Bar Chart

When to use

Categorical comparison with hover tooltips. Supports grouped and stacked modes easily.

■ Avoid when

Quick static analysis — matplotlib bar is faster.

↔ Equivalent

plt.bar() · sns.barplot()

```
fig = px.bar(df, x='city', y='sales', color='year', barmode='group',
text_auto=True) # text_auto shows values on bars
```

px.scatter() — Interactive Scatter Plot

When to use

2–5 variable relationships using x, y, color, size, symbol — all interactive.

■ Avoid when

Large datasets >50k points — use px.density_heatmap() or hexbin.

↔ Equivalent

plt.scatter() · sns.scatterplot()

```
fig = px.scatter(df, x='gdp', y='life_exp', size='population',
color='continent', hover_name='country', log_x=True)
```

px.histogram() — Interactive Histogram

When to use

Distribution with interactive bin adjustment. Supports marginal plots (box/violin/rug).

■ Avoid when

When you need KDE overlay — sns.histplot(kde=True) is more flexible.

↔ Equivalent

plt.hist() · sns.histplot()

```
fig = px.histogram(df, x='age', color='gender', nbins=40, marginal='box',
barmode='overlay', opacity=0.7)
```

px.box() — Interactive Box Plot

When to use

Median/IQR/outliers with hover showing exact values. Easier to explore than static.

■ Avoid when

Need distribution shape — use px.violin().

↔ Equivalent

plt.boxplot() · sns.boxplot()

```
fig = px.box(df, x='department', y='salary', color='level',
points='outliers', notched=True) # notched=True shows CI on median
```


px.violin() — Interactive Violin Plot

When to use

Distribution shape + box plot stats combined. Great for comparing group distributions interactively.

■ Avoid when

Small datasets (<30) — shape is unreliable.

↔ Equivalent

`sns.violinplot()` · (matplotlib: manual)

```
fig = px.violin(df, x='class', y='age', color='sex', box=True,
points='all') # box=True adds inner boxplot
```

px.pie() / px.sunburst() — Pie & Sunburst

When to use

Pie for simple proportions. Sunburst for hierarchical part-of-whole (multi-level donut).

■ Avoid when

More than 6 slices for pie. Sunburst needs clear hierarchy.

↔ Equivalent

`plt.pie()` · (seaborn: no native pie)

```
fig = px.pie(df, names='category', values='amount', hole=0.4) # hole=donut
# Sunburst: px.sunburst(df, path=['region', 'country'], values='sales')
```

px.treemap() — Treemap

When to use

Hierarchical proportions as nested rectangles. Better than sunburst for many items.

■ Avoid when

Non-hierarchical data — use bar chart.

↔ Equivalent

(matplotlib: squarify library) · (seaborn: no native)

```
fig = px.treemap(df, path=[px.Constant('All'), 'department', 'team'],
values='budget', color='growth', color_continuous_scale='RdYlGn')
```

px.area() — Stacked Area Chart

When to use

Show how multiple series add up to a total over time. Interactive version of stackplot.

■ Avoid when

Negative values or many overlapping series.

↔ Equivalent

`plt.stackplot()` · (seaborn: manual)

```
fig = px.area(df, x='year', y='value', color='source',
title='Energy Sources Over Time')
```

px.heatmap() via px.imshow() — Heatmap

When to use

Interactive color matrix with hover showing exact values. Great for correlation/confusion matrix.

■ Avoid when

Small matrices where `sns.heatmap(annot=True)` is more readable.

↔ Equivalent

`sns.heatmap()` · `plt.imshow()`

```
import plotly.express as px
fig = px.imshow(df.corr(), color_continuous_scale='RdBu_r', zmin=-1, zmax=1,
text_auto='.2f') # text_auto annotates each cell
```

px.scatter_matrix() — Pair Plot

When to use

Interactive version of seaborn pairplot.
Click/brush to filter across all subplots.

■ Avoid when

More than 7 features — too small.
Static reports — use `sns.pairplot()`.

↔ Equivalent

`sns.pairplot()` · (matplotlib: manual)

```
fig = px.scatter_matrix(df, dimensions=['col1', 'col2', 'col3'],
color='species', opacity=0.7)
```

px.choropleth() — Choropleth Map

When to use

Color geographic regions by a numeric value. Built-in world/US maps.

■ Avoid when

Point data or city-level data — use `px.scatter_geo()` instead.

↔ Equivalent

(matplotlib: `geopandas+matplotlib`) · (seaborn: no native)

```
fig = px.choropleth(df, locations='country_code', color='gdp',
locationmode='ISO-3', color_continuous_scale='Viridis')
```

px.scatter_geo() / px.scatter_mapbox() — Geo Scatter

When to use

Plot points on a map. `scatter_geo` for world maps, `scatter_mapbox` for tile-based street maps.

■ Avoid when

Regional/country aggregates — use `choropleth`.

↔ Equivalent

(matplotlib: `basemap/cartopy`) · (seaborn: no native)

```
fig = px.scatter_mapbox(df, lat='lat', lon='lon', size='magnitude',
color='depth', mapbox_style='carto-positron', zoom=3)
```

px.funnel() — Funnel Chart

When to use

Show sequential drop-off stages. Built-in in Plotly — not available natively in mpl/sns.

■ Avoid when

Non-sequential stages or circular flow — use Sankey.

↔ Equivalent

(matplotlib: manual bars) · (seaborn: no native)

```
fig = px.funnel(df, x='count', y='stage',
title='Conversion Funnel: Visit → Purchase')
```

go.Sankey() — Sankey Diagram

When to use

Flow and quantity between nodes/stages. Requires `plotly.graph_objects` (not `express`).

■ Avoid when

Too many nodes — becomes unreadable spaghetti.

↔ Equivalent

(matplotlib: `pySankey` library) · (seaborn: no native)

```
import plotly.graph_objects as go
fig = go.Figure(go.Sankey(
node=dict(label=['A', 'B', 'C']),
link=dict(source=[0,1], target=[1,2], value=[8,4])))
fig.show()
```

go.Candlestick() — Candlestick Chart

When to use

OHLC financial data over time. Shows open, high, low, close per period.

■ Avoid when

Non-financial time series — use line chart.

↔ Equivalent

(matplotlib: mplfinance library) · (seaborn: no native)

```
import plotly.graph_objects as go
fig = go.Figure(go.Candlestick(x=df['date'],
open=df['open'], high=df['high'], low=df['low'], close=df['close']))
fig.show()
```

go.Surface() / go.Scatter3d() — 3D Plots

When to use

Visualize 3D data interactively. Rotate/zoom in browser. For ML surfaces, math functions.

■ Avoid when

Static reports or simple 2D data — adds complexity without value.

↔ Equivalent

(matplotlib: mpl_toolkits.mplot3d) · (seaborn: no native 3D)

```
fig = go.Figure(go.Surface(z=Z_matrix, x=X, y=Y, colorscale='Viridis'))
# Scatter3d: go.Scatter3d(x=xs, y=ys, z=zs, mode='markers')
```

Quick Decision Guide — Which Library + Chart?

Goal / Situation	Matplotlib	Seaborn	Plotly
Time series / trend	<code>plt.plot()</code>	<code>sns.lineplot()</code>	<code>px.line()</code>
Compare categories	<code>plt.bar()</code>	<code>sns.barplot()</code>	<code>px.bar()</code>
Count frequencies	<code>plt.bar()</code> + <code>value_counts</code>	<code>sns.countplot()</code>	<code>px.histogram()</code>
Distribution (one var)	<code>plt.hist()</code>	<code>sns.histplot(kde=True)</code>	<code>px.histogram(marginal='box')</code>
Distribution shape	<code>plt.hist()</code> + KDE manual	<code>sns.kdeplot()</code>	<code>px.violin()</code>
Spread + outliers	<code>plt.boxplot()</code>	<code>sns.boxplot()</code>	<code>px.box()</code>
Full dist shape + stats	Manual	<code>sns.violinplot()</code>	<code>px.violin(box=True)</code>
2 variable relationship	<code>plt.scatter()</code>	<code>sns.scatterplot()</code>	<code>px.scatter()</code>
3 variable relationship	<code>plt.scatter(c=, s=)</code>	<code>sns.scatterplot(size=)</code>	<code>px.scatter(size=, color=)</code>
Correlation matrix	<code>plt.imshow()</code>	<code>sns.heatmap(annot=True)</code>	<code>px.imshow(text_auto=True)</code>
All variables vs all (EDA)	Manual subplots	<code>sns.pairplot()</code>	<code>px.scatter_matrix()</code>
Regression line	<code>np.polyfit</code> + <code>plt.plot()</code>	<code>sns.regplot()</code>	<code>px.scatter(trendline='ols')</code>
Part of whole (simple)	<code>plt.pie()</code>	— (no native pie)	<code>px.pie()</code>
Part of whole (hierarchy)	— (squarify lib)	— (no native)	<code>px.treemap()</code> / <code>px.sunburst()</code>
Stacked over time	<code>plt.stackplot()</code>	Manual <code>fill_between</code>	<code>px.area()</code>
Data on map (regions)	— (geopandas)	— (no native)	<code>px.choropleth()</code>
Data on map (points)	— (cartopy/basemap)	— (no native)	<code>px.scatter_mapbox()</code>
Conversion funnel	Manual bars	— (no native)	<code>px.funnel()</code>
Flow between stages	— (pySankey lib)	— (no native)	<code>go.Sankey()</code>
Financial OHLC	— (mplfinance lib)	— (no native)	<code>go.Candlestick()</code>
3D surface / scatter	<code>mpl_toolkits Axes3D</code>	— (no native 3D)	<code>go.Surface()</code> / <code>go.Scatter3d()</code>
Grid of subplots by group	<code>plt.subplots()</code>	<code>sns.FacetGrid()</code>	<code>px.facet_col</code> / <code>facet_row</code>
Joint bivariate + marginals	Manual <code>GridSpec</code>	<code>sns.jointplot()</code>	<code>px.scatter(marginal_x=)</code>
Cluster + heatmap	Manual dendrograms	<code>sns.clustermap()</code>	<code>px.imshow()</code> + <code>scipy cluster</code>

Matplotlib = control · Seaborn = statistics · Plotly = interactivity | — means not natively supported (use another lib or manual workaround)